

KoiCloud — Propuesta de Proyecto (alcance reducido)

Proyecto Final — Ingeniería de Software I (2026), Universidad Rafael Landívar

Categoría: Base de Datos como Servicio (DBaaS)

Versión: 3 — 12 de septiembre de 2026

Motivo de la revisión: observaciones del catedrático + realidad de ejecución en solitario

1. Resumen ejecutivo

KoiCloud es una plataforma DBaaS: un desarrollador se registra, contrata un plan con pago simulado y obtiene una instancia PostgreSQL gestionada (*pond*) sin operar infraestructura.

Compromiso del semestre: una **interfaz web** completa sobre un control plane FastAPI y un node-agent que aprovisiona contenedores Docker reales en un VPS. Incluye consola SQL en el navegador, respaldos básicos y panel de administración.

No forma parte del compromiso: CLI, servidor MCP / agentes de IA, auto-curación como producto, ni validación de hipótesis comerciales con usuarios reales.

2. Problemas que sí resolvemos

#	Problema (operación, verificable en el semestre)	Evidencia
1	Crear Postgres exige conocimiento de ops	URI utilizable tras “crear pond”
2	El estado del servicio es opaco	Panel con estado del contenedor
3	Suscripción y pago académicos	Flujo simulado + historial + factura
4	No hay cliente SQL a mano	Consola web con resultados tabulares

Hipótesis de negocio (fuera de lo demostrable)

El hueco de precio frente a ~USD 25/mes y la factura con NIT orientan el catálogo, pero **requieren usuarios reales**. Quedan como hipótesis, no como objetivos del proyecto.

3. Catálogo de servicios

Plan	Descripción	Precio	Vigencia
Sandbox	Uso interno / pruebas del producto	USD 0	10 minutos
Micro	1 pond, 1 GB, backups 7 días	USD 5.00 / mes (IVA incluido)	Mensual
Pro	Hasta 10 ponds, post-pago	USD 0.02 / h + almacenamiento	Por uso

Pago: **simulado** (enunciado). Selección de plan, factura con desglose de IVA, historial, renovación y cancelación: implementados.

4. Alcance comprometido (sin niveles)

No hay núcleo / deseable / ambicioso. Esta es la lista completa:

1. Auth: registro, verificación de correo, recuperación, JWT+refresh; roles Cliente y Administrador.
2. Suscripciones: catálogo, contratación simulada, historial, renovación, cancelación.
3. DBaaS: crear / estado / conexión / eliminar PostgreSQL 16 en Docker.
4. Consola SQL en el navegador (lectura por defecto).
5. Respallos diarios y restauración bajo demanda.
6. Vista de uso (horas y almacenamiento).
7. Panel administrador.
8. Arquitectura control plane + node-agent + cola en PostgreSQL; un nodo.

Fuera de alcance

- CLI e IaC declarativa.
- MCP, llaves de agente, cobro por operación de agente.
- Auto-curación como garantía o demo de caos.

- Validación empírica de precio/NIT.
- Otros motores, branching, réplicas, pooler, extensiones.
- Organizaciones/equipos.
- Cobro real, pasarelas, SAT, SLA.
- HA, multi-nodo.
- Roles Soporte / Operador; status page pública; sandbox público sin registro.
- 2FA; OAuth 2.1 para agentes.

5. Arquitectura (defendible con el conocimiento del equipo)

Monolito modular FastAPI que escribe estado deseado; node-agent ejecuta Docker y reporta estado observado; cola de jobs en PostgreSQL (SKIP LOCKED). Un VPS.

Descartado: microservicios (costo operativo sin ganancia con un desarrollador); Kubernetes (oculta el provisioning y nadie lo opera); CLI/MCP en v1 (segunda y tercera superficie no caben en un semestre en solitario).

Detalle de diagramas: paquete docs/entrega-2/.

6. Stack

React + Vite + TypeScript · Python 3.12 + FastAPI · PostgreSQL 16 · Docker · Caddy · JWT/argon2 · Resend o console para correo.

7. Metodología y cronograma (ajustado a un ejecutor)

Scrum académico en sprints alineados a los hitos del curso. Sin asumir paralelismo de cuatro roles: el mismo ejecutor prioriza el camino crítico (auth → ponds reales → billing → consola → docs).

Hito	Fecha	Objetivo
Propuesta	28 ago	v1

Requisitos y diseño	11 sep	Entrega 2 (este alcance)
Avance 30 %	25 sep	Auth + planes + 1 pond real
Avance 50 %	9 oct	Billing + backups + uso
Avance 80 %	23 oct	Consola SQL + admin + pulido
Final	1ª sem nov	Docs + demo + exposición

8. Riesgos

Riesgo	Mitigación
Trabajo en solitario	Alcance ya recortado; mock del agente para desarrollar sin VPS
VPS cae el día de la demo	Video de respaldo + modo mock
Sobreingeniería residual del harness	Entrega 2 y esta propuesta mandan sobre tickets ambiciosos del harness
Exponer Postgres a Internet	Contraseñas fuertes, puertos altos, firewall

9. Por qué este alcance es honesto

Cumple el enunciado (usuarios, pagos simulados, catálogo DBaaS, web, arquitectura defendida). Incorpora el feedback del catedrático sin fingir un equipo de cuatro. Lo que impresiona en la demo — contenedor real y consola SQL— no depende de MCP ni de auto-curación.